

Absolutely. The main idea of **ICEPOT** is to convert each weak testing request into a **clear, context-rich prompt** that gives the AI enough information to generate useful test cases.

Below I've filled **I-C-E-P-O-T** for all 7 scenarios, using QA/automation-oriented examples.

1. Search Functionality

I — Instruction

Test the search functionality comprehensively, including positive, negative, boundary, filter, sort, and performance scenarios.

C — Context

The application is an **e-commerce website** where users can search for products by product name or keyword.

- Search should return relevant products.
- Users can apply filters such as category, price range, brand, and rating.
- Users can sort results by price, rating, and relevance.
- Empty and invalid searches should be handled appropriately.
- Search results should load within an acceptable response time.

E — Example

For example:

- Search for "iPhone" → relevant products should be displayed.
- Search with an empty value → appropriate validation should be displayed.
- Search for "@#\$\$%" → system should handle invalid input gracefully.
- Search with a keyword that has no matching products → "No products found" should be displayed.
- Apply price and brand filters → only matching products should appear.
- Sort results by price → products should be displayed in the correct order.
- Search response should be within **2 seconds** under normal load.

P — Persona

Act as a **Senior QA Engineer with experience in web application and e-commerce testing**.

O — Output Format

Provide test cases in a table with:

TC ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result	Priority	Test Type
-------	---------------	---------------	------------	-----------	-----------------	----------	-----------

Include functional, negative, boundary, usability, and performance scenarios.

T — Tone

Professional, precise, structured, and QA-focused.

2. File Upload

I — Instruction

Generate comprehensive test cases for the file upload functionality, covering valid, invalid, boundary, security, and failure scenarios.

C — Context

The application allows users to upload documents.

Supported file types:

- PDF
- DOC
- DOCX
- XLS
- XLSX
- JPG
- PNG

Maximum allowed file size is **10 MB**.

The application should display a success message after a valid upload and an appropriate error message for invalid files.

Security testing should also verify that malicious or executable files cannot be uploaded.

E — Example

For example:

- Upload a valid 5 MB PDF → upload should succeed.
- Upload a 10 MB file → upload should succeed.
- Upload a file greater than 10 MB → upload should fail.
- Upload `.exe` file → upload should be rejected.
- Upload unsupported `.zip` file → appropriate error should be displayed.
- Upload corrupted PDF → upload should fail gracefully.
- Upload file with special characters in the filename → system should handle it correctly.
- Interrupt network connection during upload → appropriate error should be displayed.
- Attempt malicious file upload → application should reject it.

P — Persona

Act as a **Senior QA Engineer specializing in web application, security, and file-upload testing.**

O — Output Format

Create a table containing:

TC ID	Scenario	Preconditions	Test Steps	File/Test Data	Expected Result	Priority	Test Type
-------	----------	---------------	------------	----------------	-----------------	----------	-----------

Categorize scenarios as Functional, Negative, Boundary, Security, and Performance.

T — Tone

Professional, security-aware, detailed, and concise.

3. Shopping Cart

I — Instruction

Generate comprehensive test cases for shopping cart functionality, covering product addition, removal, quantity updates, pricing, discounts, inventory, and checkout behavior.

C — Context

The application is an **e-commerce website**.

Users can:

- Add products to the cart.
- Remove products.
- Increase/decrease quantity.
- View subtotal.
- Apply coupon codes.
- View discounts.
- View taxes and shipping charges.
- Proceed to checkout.

The cart should correctly handle out-of-stock products and inventory changes.

E — Example

For example:

- Add one product → product should appear in cart.
- Add multiple products → all products should appear correctly.
- Increase quantity from 1 to 3 → subtotal should be recalculated.
- Remove product → product should disappear and total should update.
- Apply valid coupon → discount should be calculated correctly.
- Apply expired coupon → appropriate error should appear.
- Add an out-of-stock product → product should not be added.
- Product becomes out of stock after being added → cart should notify the user.
- Verify subtotal + tax + shipping – discount = final amount.
- Refresh browser → cart should retain products if persistence is supported.

P — Persona

Act as a **Senior QA Engineer experienced in e-commerce and payment-related testing**.

O — Output Format

Provide:

TC ID	Test Scenario	Preconditions	Steps	Test Data	Expected Result	Priority	Test Type
-------	---------------	---------------	-------	-----------	-----------------	----------	-----------

Include functional, calculation, boundary, negative, inventory, and usability scenarios.

T — Tone

Professional, analytical, and detail-oriented.

4. Role-Based Access Control

I — Instruction

Create comprehensive test cases to validate Role-Based Access Control and ensure that users can access only the features and data permitted for their assigned roles.

C — Context

The application is an **enterprise business application** with three roles:

- **Admin** — full access.
- **Manager** — can create, edit, approve, and view records.
- **Viewer** — read-only access.

Example permissions:

Function	Admin	Manager	Viewer
View records	Yes	Yes	Yes
Create records	Yes	Yes	No
Edit records	Yes	Yes	No
Delete records	Yes	No	No
Approve records	Yes	Yes	No
Manage users	Yes	No	No

Unauthorized users should not be able to access restricted pages through direct URLs or APIs.

E — Example

For example:

- Admin logs in → all features should be accessible.
- Manager logs in → user-management option should not be available.
- Viewer logs in → Create/Edit/Delete buttons should not be displayed.
- Viewer attempts to access an Admin URL directly → access should be denied.
- Viewer sends an unauthorized API request → API should return an appropriate authorization response.
- User changes from Viewer to Manager → permissions should update appropriately after re-login/session refresh.
- Disabled user attempts login → access should be denied.

P — Persona

Act as a **Senior QA Engineer specializing in RBAC, authorization, API, and security testing.**

O — Output Format

Provide:

TC ID	Role	Permission	Test Scenario	Steps	Expected Result	Priority	Test Type
-------	------	------------	---------------	-------	-----------------	----------	-----------

Include UI authorization, URL authorization, API authorization, positive, negative, and security scenarios.

T — Tone

Professional, security-focused, precise, and systematic.

5. API Testing — User Registration

I — Instruction

Generate comprehensive API test cases for the user registration endpoint, covering request validation, response validation, authentication, status codes, database behavior, security, and negative scenarios.

C — Context

The application exposes a **REST API** for user registration.

Endpoint:

POST /api/users/register

Example request:

```
JSON
{
  "firstName": "John",
  "lastName": "Smith",
  "email": "john.smith@test.com",
  "password": "Test@12345"
}
```

Expected successful response:

- HTTP Status: 201 Created
- User ID should be generated.
- Response should contain the registered user's basic information.
- Password should never be returned in the response.

Expected validation errors should use appropriate 4xx status codes.

Testing can be performed using **Postman** or **C# HttpClient/RestSharp**.

E — Example

For example:

- Valid registration → 201 Created .
- Duplicate email → appropriate 409 Conflict or defined business error.
- Missing email → 400 Bad Request .
- Invalid email format → validation error.
- Weak password → validation error.
- Missing mandatory fields → validation error.
- Extremely long input → boundary validation.
- SQL injection strings → request should be safely handled.
- Password should not appear in the response.
- Verify user is actually created in the database if database validation is available.

P — Persona

Act as a **Senior API Automation QA Engineer experienced in REST APIs, Postman, C# RestSharp, and HttpClient**.

O — Output Format

Provide:

TC ID	Scenario	HTTP Method	Endpoint	Request Data	Expected Status	Expected Response	Priority
-------	----------	-------------	----------	--------------	-----------------	-------------------	----------

Include functional, negative, boundary, security, API contract, and database validation scenarios.

T — Tone

Technical, precise, API-focused, and automation-ready.

6. Mobile App — OTP Login

I — Instruction

Generate comprehensive test cases for OTP-based mobile login, covering OTP generation, validation, expiry, resend, incorrect OTP, network failures, security, and platform-specific behavior.

C — Context

The application is a **mobile banking application** available on:

- Android
- iOS

Users log in using their registered mobile number and a one-time password (OTP).

Requirements:

- OTP contains **6 digits**.
- OTP expires after **5 minutes**.
- User can request a new OTP after **30 seconds**.
- Maximum **3 incorrect OTP attempts** are allowed.
- OTP should be invalidated after successful login.
- Login requires an active internet connection.

Testing will be performed using **Appium** on real Android and iOS devices.

E — Example

For example:

- Enter valid mobile number → OTP should be generated.
- Enter correct OTP → user should successfully log in.
- Enter incorrect OTP → appropriate error should appear.
- Enter expired OTP → login should fail.
- Request resend before 30 seconds → resend should not be allowed.
- Request resend after 30 seconds → new OTP should be generated.
- Enter incorrect OTP 3 times → account/login attempt should be temporarily restricted according to requirements.
- Disconnect internet during OTP verification → appropriate error should appear.
- Use old OTP after requesting a new OTP → old OTP should be invalid.
- Verify OTP cannot be reused after successful login.

P — Persona

Act as a **Senior Mobile QA Automation Engineer experienced in Appium, Android, iOS, and authentication testing**.

O — Output Format

Provide:

TC ID	Platform	Scenario	Preconditions	Steps	Test Data	Expected Result	Priorities
-------	----------	----------	---------------	-------	-----------	-----------------	------------

Cover functional, negative, boundary, network, security, usability, Android, and iOS scenarios.

T — Tone

Professional, security-conscious, mobile-testing focused, and structured.

7. Dashboard / Report Generation

I — Instruction

Generate comprehensive test cases for the report generation module, covering report types, filters, data accuracy, export functionality, permissions, and performance.

C — Context

The application provides business reports that users can generate and download in:

- PDF
- Excel
- CSV

Users can filter reports using:

- Start Date
- End Date
- Department
- Report Type
- Status

There are different user roles, and not every user has permission to view all reports.

The generated report must accurately match the data displayed on the dashboard.

E — Example

For example:

- Generate PDF report → PDF should be generated successfully.
- Generate Excel report → `.xlsx` file should download successfully.
- Generate CSV report → CSV should contain the expected records.
- Select a valid date range → only records within that range should appear.
- Start date greater than end date → validation should be displayed.
- Apply Department filter → only that department's data should appear.
- Compare dashboard count with report count → values should match.
- Viewer attempts to access restricted report → access should be denied.
- Generate report containing 100,000 records → report should complete within the defined performance threshold.
- Verify exported data is not truncated or incorrectly formatted.

P — Persona

Act as a **Senior QA Engineer specializing in dashboard, reporting, data validation, and authorization testing.**

O — Output Format

Provide:

TC ID	Report Type	Scenario	Filters/Test Data	Steps	Expected Result	Priority	Test Type
-------	-------------	----------	-------------------	-------	-----------------	----------	-----------

Include functional, data accuracy, filter, export, permission, boundary, and performance scenarios.

T — Tone

Professional, analytical, data-focused, and structured.

ICEPOT Summary

You can remember the purpose of each ICEPOT component like this:

Feature	Purpose	Question it answers
I — Instruction	What should AI do?	What do I want?
C — Context	Background and requirements	What does AI need to know?

Feature	Purpose	Question it answers
E — Example	Demonstrates expected scenarios/output	Can I show an example?
P — Persona	Defines expertise/role	Who should AI act as?
O — Output Format	Defines response structure	How should the answer look?
T — Tone	Defines communication style	How should it communicate?

A simple formula for your QA prompts

Weak prompt:

"Generate test cases for file upload."

ICEPOT prompt:

Instruction: Generate comprehensive file-upload test cases.

Context: Supported files are PDF/DOC/DOCX/XLS/XLSX, maximum size 10 MB, with security validation.

Example: Valid file, oversized file, unsupported file, corrupted file, malicious file, network failure.

Persona: Senior QA Engineer specializing in web and security testing.

Output: Test-case table with ID, scenario, steps, data, expected result, priority, and test type.

Tone: Professional and QA-focused.

This is the key benefit of **ICEPOT**: it transforms a vague instruction into a **specific, testable, and automation-ready prompt**.

Is this conversation helpful so far?